

ALGORITHM:

Main Algorithm

1. Start.
2. Display the menu:
 - Matrix addition associativity
 - Matrix multiplication associativity
 - Exit
3. Read the user's choice.
4. Use switch:
 - Choice 1 → call additionAssociative().
 - Choice 2 → call multiplicationAssociative().
 - Choice 3 → exit.
 - Otherwise → display an invalid-choice message.
5. Repeat until the user selects Exit.
6. Stop.

For Matrix Addition

1. Read positive values of m and n.
2. Declare three $m \times n$ matrices A, B, and C.
3. Read all three matrices.
4. Calculate $A+B$, then $(A+B)+C$.
5. Calculate $B+C$, then $A+(B+C)$.
6. Display both resulting matrices.
7. Compare corresponding elements.
8. If they are equal, the associative property is verified.

For Matrix Multiplication

1. Read positive values m, n, p, and q.
2. Declare $A[m][n]$, $B[n][p]$, and $C[p][q]$.

3. Read the three matrices.
4. Calculate $AB = A \times B$, then $(AB) \times C$.
5. Calculate $BC = B \times C$, then $A \times (BC)$.
6. Display both resulting $m \times q$ matrices.
7. Compare them.
8. If equal, the associative property is verified.

C PROGRAM:

```
#include <stdio.h>
```

```
/* Function prototypes */
```

```
void additionAssociative(void);
```

```
void multiplicationAssociative(void);
```

```
void readMatrix(int r, int c, int a[r][c]);
```

```
void printMatrix(int r, int c, int a[r][c]);
```

```
void addMatrix(int r, int c,  
               int a[r][c], int b[r][c], int result[r][c]);
```

```
void multiplyMatrix(int r1, int c1, int c2,  
                   int a[r1][c1], int b[c1][c2],  
                   int result[r1][c2]);
```

```
int compareMatrix(int r, int c,  
                  int a[r][c], int b[r][c]);
```

```
int main(void)
```

```
{
    int choice;

    do
    {
        printf("\n===== MATRIX MENU =====\n");
        printf("1. Verify associative property of matrix addition\n");
        printf("2. Verify associative property of matrix multiplication\n");
        printf("3. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
                additionAssociative();
                break;

            case 2:
                multiplicationAssociative();
                break;

            case 3:
                printf("Exiting program...\n");
                break;

            default:
                printf("Invalid choice! Please enter 1, 2, or 3.\n");
```

```
}
```

```
} while (choice != 3);
```

```
return 0;
```

```
}
```

```
/* Function to verify  $(A+B)+C = A+(B+C)$  */
```

```
void additionAssociative(void)
```

```
{
```

```
    int m, n;
```

```
    printf("\nEnter number of rows (m): ");
```

```
    scanf("%d", &m);
```

```
    printf("Enter number of columns (n): ");
```

```
    scanf("%d", &n);
```

```
    if (m <= 0 || n <= 0)
```

```
    {
```

```
        printf("Rows and columns must be positive.\n");
```

```
        return;
```

```
    }
```

```
/* Variable-length arrays */
```

```
int A[m][n], B[m][n], C[m][n];
```

```
int AB[m][n], BC[m][n];
```

```
int left[m][n], right[m][n];
```

```
printf("\nEnter elements of matrix A:\n");
```

```
readMatrix(m, n, A);
```

```
printf("\nEnter elements of matrix B:\n");
```

```
readMatrix(m, n, B);
```

```
printf("\nEnter elements of matrix C:\n");
```

```
readMatrix(m, n, C);
```

```
/* Left side: (A+B)+C */
```

```
addMatrix(m, n, A, B, AB);
```

```
addMatrix(m, n, AB, C, left);
```

```
/* Right side: A+(B+C) */
```

```
addMatrix(m, n, B, C, BC);
```

```
addMatrix(m, n, A, BC, right);
```

```
printf("\n(A + B) + C =\n");
```

```
printMatrix(m, n, left);
```

```
printf("\nA + (B + C) =\n");
```

```
printMatrix(m, n, right);
```

```
if (compareMatrix(m, n, left, right))
```

```
    printf("\nAssociative property of matrix addition is VERIFIED.\n");
```

```
else
```

```
    printf("\nAssociative property of matrix addition is NOT verified.\n");  
}
```

```
/* Function to verify  $(A*B)*C = A*(B*C)$  */
```

```
void multiplicationAssociative(void)
```

```
{
```

```
    int m, n, p, q;
```

```
    printf("\nFor A(m x n), B(n x p), C(p x q)\n");
```

```
    printf("Enter m: ");
```

```
    scanf("%d", &m);
```

```
    printf("Enter n: ");
```

```
    scanf("%d", &n);
```

```
    printf("Enter p: ");
```

```
    scanf("%d", &p);
```

```
    printf("Enter q: ");
```

```
    scanf("%d", &q);
```

```
    if (m <= 0 || n <= 0 || p <= 0 || q <= 0)
```

```
    {
```

```
        printf("All dimensions must be positive.\n");
```

```
        return;
```

```
    }
```

```
/* Variable-length arrays */
```

```
int A[m][n];
```

```
int B[n][p];
```

```
int C[p][q];
```

```
int AB[m][p];
```

```
int BC[n][q];
```

```
int left[m][q];
```

```
int right[m][q];
```

```
printf("\nEnter elements of matrix A (%d x %d):\n", m, n);
```

```
readMatrix(m, n, A);
```

```
printf("\nEnter elements of matrix B (%d x %d):\n", n, p);
```

```
readMatrix(n, p, B);
```

```
printf("\nEnter elements of matrix C (%d x %d):\n", p, q);
```

```
readMatrix(p, q, C);
```

```
/* Left side: (A x B) x C */
```

```
multiplyMatrix(m, n, p, A, B, AB);
```

```
multiplyMatrix(m, p, q, AB, C, left);
```

```
/* Right side: A x (B x C) */
```

```
multiplyMatrix(n, p, q, B, C, BC);
```

```
multiplyMatrix(m, n, q, A, BC, right);
```

```

printf("\n(A x B) x C =\n");
printMatrix(m, q, left);

printf("\nA x (B x C) =\n");
printMatrix(m, q, right);

if (compareMatrix(m, q, left, right))
    printf("\nAssociative property of matrix multiplication is VERIFIED.\n");
else
    printf("\nAssociative property of matrix multiplication is NOT verified.\n");
}

/* Function to read a matrix */
void readMatrix(int r, int c, int a[r][c])
{
    int i, j;

    for (i = 0; i < r; i++)
    {
        for (j = 0; j < c; j++)
        {
            scanf("%d", &a[i][j]);
        }
    }
}

```

```
/* Function to display a matrix */  
void printMatrix(int r, int c, int a[r][c])  
{  
    int i, j;  
  
    for (i = 0; i < r; i++)  
    {  
        for (j = 0; j < c; j++)  
        {  
            printf("%d\t", a[i][j]);  
        }  
        printf("\n");  
    }  
}
```

```
/* Function for matrix addition */  
void addMatrix(int r, int c,  
               int a[r][c], int b[r][c], int result[r][c])  
{  
    int i, j;  
  
    for (i = 0; i < r; i++)  
    {  
        for (j = 0; j < c; j++)  
        {  
            result[i][j] = a[i][j] + b[i][j];  
        }  
    }  
}
```

```
    }  
  }  
}
```

```
/* Function for matrix multiplication */
```

```
void multiplyMatrix(int r1, int c1, int c2,
```

```
    int a[r1][c1], int b[c1][c2],
```

```
    int result[r1][c2])
```

```
{
```

```
    int i, j, k;
```

```
    for (i = 0; i < r1; i++)
```

```
    {
```

```
        for (j = 0; j < c2; j++)
```

```
        {
```

```
            result[i][j] = 0;
```

```
            for (k = 0; k < c1; k++)
```

```
            {
```

```
                result[i][j] += a[i][k] * b[k][j];
```

```
            }
```

```
        }
```

```
    }
```

```
}
```

```
/* Function to compare two matrices */
```

```

int compareMatrix(int r, int c,
                  int a[r][c], int b[r][c])
{
    int i, j;

    for (i = 0; i < r; i++)
    {
        for (j = 0; j < c; j++)
        {
            if (a[i][j] != b[i][j])
                return 0;
        }
    }

    return 1;
}

```

TEST CASE TABLE:

TEST	CHOICE	DIMENSIONS	EXAMPLE INPUT	EXPECTED RESULT
1	1	m=2, n=2	Three valid 2×2 matrices	Both sides equal; addition associative property verified
2	1	m=1, n=3	Three valid 1×3 matrices	Both sides equal; property verified
3	2	m=2,n=2,p=2,q=2	Three valid 2×2 matrices	Both sides equal; multiplication associative property verified
4	2	m=2,n=3,p=2,q=1	A: 2×3, B: 3×2, C: 2×1	Both sides produce equal 2×1 matrices
5	1	m=0,n=2	Invalid dimension	Rows and columns must be positive.

6	2	m=2,n=-1,p=2,q=2	Invalid dimension	All dimensions must be positive
7	5	--	Invalid menu choice	Invalid choice! Please enter 1, 2, or 3.
8	3	--	Exit	Program terminates

```

Session: kali@kali - [~]
$ vi h33.c

(kali@kali)-[~]
$ gcc h33.c -o h33

(kali@kali)-[~]
$ ./h33

===== MATRIX MENU =====
1. Verify associative property of matrix addition
2. Verify associative property of matrix multiplication
3. Exit
Enter your choice: 1

Enter number of rows (m): 2
Enter number of columns (n): 2

Enter elements of matrix A:
1 2 3 4

Enter elements of matrix B:
4 5 6 7

Enter elements of matrix C:
9 8 7 6

(A + B) + C =
14      15
16      17

A + (B + C) =
14      15
16      17

Associative property of matrix addition is VERIFIED.

```